exercisesweek36

September 3, 2026

1 Exercises week 36

FYS-STK3155/4155, fall 2026 — deadline Friday September 4 at midnight.

The aim this week is that you own the two central results of Monday's lecture — the statistics of the OLS estimator and the bias-variance decomposition, Eq. (2.47) of the book — and the two resampling tools, the bootstrap and k-fold cross-validation. The code you developed in the Tuesday session (`week36tuesday.ipynb`) is meant to be reused here, and everything in this set is directly reusable in Project 1.

Reading: Chapter 2, Sections 2.3–2.12 of the book; Hastie et al., Sections 7.1–7.5, 7.10 and 7.11.

1.1 Exercise 1: Analytical exercises — the statistics of the OLS estimator

Assume the data are generated by the linear model of Eq. (2.37), $y = X\theta + \varepsilon$, with independent noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and a fixed (non-stochastic) design matrix X of dimension $n \times p$.

1a) Show that the expectation value and variance of a single output are (Eq. (2.39))

$$\mathbb{E}[y_i] = X_{i,*}\,\theta, \qquad \mathrm{Var}(y_i) = \sigma^2.$$

1b) Show that the OLS estimator is unbiased, $\mathbb{E}[\hat{\theta}] = \theta$ (Eq. (2.40)), and derive its variance (Eq. (2.42)),

$$\mathrm{Var}(\hat{\theta}) = \sigma^2 \left(X^T X\right)^{-1}.$$

Hint: use $\mathbb{E}[yy^T] = X\theta\theta^T X^T + \sigma^2 I_{nn}$.

1c) Optional: Explain how the diagonal elements of $\sigma^2(X^TX)^{-1}$ give a confidence interval for the coefficient θ_j (Eq. (2.43)), and why in practice σ^2 is replaced by $\hat{\sigma}^2 = \|y - X\hat{\theta}\|_2^2/(n-p)$. Where does the $n-p$ come from? (Think of Bessel's correction, Eq. (2.28).)

1.2 Solution exercise 1a)

We assume that the error term ϵ is normally distributed with mean zero and variance σ^2, so

$$\epsilon \sim \mathcal{N}(0, \sigma^2), \qquad p(\epsilon) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right).$$

The linear regression model is $y = X\theta + \epsilon$. Using the linearity of expectation,

$$\mathbb{E}[y] = \mathbb{E}[X\theta + \epsilon] = \mathbb{E}[X\theta] + \mathbb{E}[\epsilon].$$

Since X and θ are deterministic, $\mathbb{E}[X\theta] = X\theta$. Furthermore, the error term has zero mean,

$$\mathbb{E}[\epsilon] = \int_{-\infty}^{\infty} \epsilon\, p(\epsilon)\, d\epsilon = 0,$$

because $\epsilon p(\epsilon)$ is an odd function integrated over the symmetric interval $(-\infty, \infty)$. Therefore,

$$\mathbb{E}[y] = X\theta.$$

For a single observation,

$$y_i = X_{i,*}\theta + \epsilon_i,$$

where $X_{i,*}$ denotes the i-th row of the design matrix. Its expectation value is therefore

$$\mathbb{E}[y_i] = X_{i,*}\theta.$$

To determine the variance of y_i, we start from the definition

$$\mathrm{Var}(y_i) = \mathbb{E}\left[\left(y_i - \mathbb{E}[y_i]\right)^2\right].$$

Substituting the expression for y_i,

$$\mathrm{Var}(y_i) = \mathbb{E}\left[\left(X_{i,*}\theta + \epsilon_i - X_{i,*}\theta\right)^2\right] = \mathbb{E}[\epsilon_i^2].$$

Because $\mathbb{E}[\epsilon_i] = 0$, we have

$$\mathrm{Var}(\epsilon_i) = \mathbb{E}[\epsilon_i^2] = \sigma^2.$$

This can also be shown directly from the Gaussian probability density by integration. Dropping the subscript i for simplicity,

$$\mathbb{E}[\epsilon^2] = \int_{-\infty}^{\infty} \epsilon^2 p(\epsilon)\, d\epsilon,$$

which gives

$$\mathbb{E}[\epsilon^2] = \int_{-\infty}^{\infty} \epsilon^2 \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)\, d\epsilon.$$

Introducing the substitution

$$z = \frac{\epsilon}{\sigma}, \qquad \epsilon = \sigma z, \qquad d\epsilon = \sigma\, dz,$$

we obtain

$$\mathbb{E}[\epsilon^2] = \frac{\sigma^2}{\sqrt{2\pi}} \int_{-\infty}^{\infty} z^2 e^{-z^2/2}\, dz = \frac{\sigma^2}{\sqrt{2\pi}} \sqrt{2\pi} = \sigma^2.$$

Hence, $\mathrm{Var}(y_i) = \sigma^2$. Thus, each output y_i has mean

$$\boxed{\mathbb{E}[y_i] = X_{i,*}\theta}$$

and variance

$$\boxed{\mathrm{Var}(y_i) = \sigma^2}$$

1.3 Solution exercise 1b)

We want to show that the OLS estimator is unbiased. The least-squares estimator for θ is

$$\hat{\theta} = (X^T X)^{-1} X^T y.$$

The expectation value of $\hat{\theta}$ is

$$\mathbb{E}[\hat{\theta}] = \mathbb{E}\left[(X^T X)^{-1} X^T y\right] = (X^T X)^{-1} X^T \mathbb{E}[y],$$

which is allowed because $(X^T X)^{-1} X^T$ does not depend on y. From 1a) we have $\mathbb{E}[y] = X\theta$, so

$$\mathbb{E}[\hat{\theta}] = (X^T X)^{-1} X^T X\theta = \theta,$$

where we have used that $(X^T X)^{-1}(X^T X) = I_{nn}$. Thus, we have shown that the OLS estimator is unbiased.

We now calculate the variance of $\hat{\theta}$,

$$\mathrm{Var}(\hat{\theta}) = \mathbb{E}\left[\hat{\theta}\hat{\theta}^T\right] - \mathbb{E}[\hat{\theta}]\mathbb{E}[\hat{\theta}]^T.$$

Utilizing that the estimator is unbiases, $\mathbb{E}[\hat{\theta}] = \theta$, and substituting $\hat{\theta} = (X^T X)^{-1} X^T y$ yields

$$\mathrm{Var}(\hat{\theta}) = (X^T X)^{-1} X^T \mathbb{E}\left[yy^T\right] X (X^T X)^{-1} - \theta\theta^T.$$

The covariance matrix of y is $\mathrm{Var}(y) = \sigma^2 I_{nn}$. Using the hint,

$$\mathbb{E}\left[yy^T\right] = X\theta\theta^T X^T + \sigma^2 I_{nn},$$

we obtain

$$\mathrm{Var}(\hat{\theta}) = (X^T X)^{-1} X^T \left(X\theta\theta^T X^T + \sigma^2 I_{nn} \right) X (X^T X)^{-1} - \theta\theta^T.$$

Expanding the expression gives

$$\begin{aligned} \mathrm{Var}(\hat{\theta}) &= \theta\theta^T + \sigma^2 (X^T X)^{-1} X^T X (X^T X)^{-1} - \theta\theta^T \\ &= \sigma^2 (X^T X)^{-1}, \end{aligned}$$

where we have used that the the X matrices become the identity and the first and last term cancels. Therefore,

$$\boxed{\mathrm{Var}(\hat{\theta}) = \sigma^2 (X^T X)^{-1}}$$

1.4 Solution exercise 1c)

In general, the standard deviation is given by $\sigma = \sqrt{\mathrm{Var}}$. Using the result from exercise 1b),

$$\mathrm{Var}(\hat{\theta}) = \sigma^2 (X^T X)^{-1},$$

the standard deviation of the j-th estimated parameter is therefore

$$\sigma(\hat{\theta}_j) = \sqrt{\sigma^2 \left[(X^T X)^{-1} \right]_{jj}} = \sigma \sqrt{\left[(X^T X)^{-1} \right]_{jj}}.$$

In practice, the true value of σ^2 is usually unknown and must be estimated from the residuals. We define the residual vector as

$$r = y - X\hat{\theta}.$$

Using $\hat{\theta} = (X^T X)^{-1} X^T y$, we obtain

$$r = y - X(X^T X)^{-1} X^T y.$$

We can write this as

$$r = My,$$

where $M = I - X(X^T X)^{-1} X^T$ is the residual projection matrix. Since $y = X\theta + \epsilon$, we get

$$r = M(X\theta + \epsilon) = MX\theta + M\epsilon.$$

Since

$$MX = \left[I - X(X^T X)^{-1} X^T\right] X = 0,$$

we obtain

$$r = M\epsilon.$$

Now consider the squared residual $r^T r$. Since M is symmetric and idempotent, that is,

$$M^T = M, \qquad M^2 = M,$$

we have

$$r^T r = \epsilon^T M^T M \epsilon = \epsilon^T M \epsilon.$$

Taking the expectation value,

$$\mathbb{E}\left[r^T r\right] = \mathbb{E}\left[\epsilon^T M \epsilon\right].$$

Using the identities

$$\mathbb{E}\left[\epsilon^T A \epsilon\right] = \mathrm{tr}\left(A \mathbb{E}[\epsilon\epsilon^T]\right), \qquad \mathbb{E}[\epsilon\epsilon^T] = \sigma^2 I,$$

we obtain

$$\mathbb{E}\left[r^T r\right] = \sigma^2 \,\mathrm{tr}(M).$$

The trace of M is

$$\mathrm{tr}(M) = \mathrm{tr}(I) - \mathrm{tr}\left[X(X^T X)^{-1} X^T\right].$$

Using the cyclic property of the trace,

$$\mathrm{tr}\left[X(X^T X)^{-1} X^T\right] = \mathrm{tr}\left[(X^T X)^{-1} X^T X\right] = \mathrm{tr}(I_{pp}) = p.$$

Since

$$\mathrm{tr}(I_{nn}) = n,$$

we get

$$\mathrm{tr}(M) = n - p.$$

Therefore,

$$\mathbb{E}\left[r^T r\right] = (n-p)\sigma^2.$$

This gives the estimator

$$\hat{\sigma}^2 = \frac{r^T r}{n-p}$$

or equivalently,

$$\hat{\sigma}^2 = \frac{\left\|y - X\hat{\theta}\right\|_2^2}{n-p},$$

where n is the number of observations and p is the number of fitted parameters. Finally, replacing the unknown σ by its estimate $\hat{\sigma}$, the estimated standard deviation of $\hat{\theta}_j$ becomes

$$\boxed{\hat{\sigma}(\hat{\theta}_j) = \hat{\sigma}\sqrt{\left[(X^T X)^{-1}\right]_{jj}}}$$

1.5 Exercise 2: Analytical exercises — deriving the bias-variance tradeoff

Assume now, as in Section 2.10, that the true data are generated by $y = f(x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ (Eq. (2.44)), and let $\tilde{y}$ be the prediction of a model fitted on a randomly drawn training set.

2a) Starting from the expected squared error $\mathbb{E}[(y - \tilde{y})^2]$ and adding and subtracting $\mathbb{E}[\tilde{y}]$, derive the decomposition of Eq. (2.47),

$$\mathbb{E}\left[(y - \tilde{y})^2\right] = \frac{1}{n}\sum_i \left(f_i - \mathbb{E}[\tilde{y}]\right)^2 + \frac{1}{n}\sum_i \left(\tilde{y}_i - \mathbb{E}[\tilde{y}]\right)^2 + \sigma^2.$$

Show explicitly that all three cross terms vanish, stating for each one which assumption kills it (zero-mean noise, independence of noise and training set, f non-stochastic).

2b) Explain in your own words what the expectation $\mathbb{E}[\cdot]$ runs over, and what each of the three terms means for the polynomial-fit example of the lecture.

2c) In the numerical estimate (Tuesday's Case 1) the bias is computed against y_{test} rather than the unknown f. Show that the measured "bias" then contains σ^2, so the printed identity `error >= bias + variance` is an inequality.

1.6 Solution exercise 2a)

Writing the expectation value in terms of a sum,

$$\mathbb{E}\left[\left(y - \tilde{y}\right)^2\right] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\left[\left(y_i - \tilde{y}_i\right)^2\right].$$

Since

$$y_i = f(x_i) + \epsilon_i,$$

we have

$$\mathbb{E}\left[(y_i - \tilde{y}_i)^2\right] = \mathbb{E}\left[(f(x_i) + \epsilon_i - \tilde{y}_i)^2\right].$$

Adding and subtracting $\mathbb{E}[\tilde{y}_i]$ gives

$$\mathbb{E}\left[(f(x_i) - \mathbb{E}[\tilde{y}_i] + \mathbb{E}[\tilde{y}_i] - \tilde{y}_i + \epsilon_i)^2\right].$$

Define

$$A = f(x_i) - \mathbb{E}[\tilde{y}_i],$$

$$B = \mathbb{E}[\tilde{y}_i] - \tilde{y}_i,$$

and

$$C = \epsilon_i.$$

Then

$$\mathbb{E}\left[(A + B + C)^2\right] = \mathbb{E}\left[A^2 + B^2 + C^2 + 2AB + 2AC + 2BC\right].$$

The first three terms are

$$\mathbb{E}[A^2] = \left(f(x_i) - \mathbb{E}[\tilde{y}_i]\right)^2,$$

$$\mathbb{E}[B^2] = \mathbb{E}\left[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2\right] = \mathrm{Var}(\tilde{y}_i),$$

and

$$\mathbb{E}[C^2] = \mathbb{E}[\epsilon_i^2] = \sigma^2.$$

For the cross terms,

$$\mathbb{E}[AB] = \mathbb{E}\left[(f(x_i) - \mathbb{E}[\tilde{y}_i])\,(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])\right].$$

Since $f(x_i) - \mathbb{E}[\tilde{y}_i]$ is not stochastic, it can be taken outside the expectation value:

$$\mathbb{E}[AB] = (f(x_i) - \mathbb{E}[\tilde{y}_i])\,\mathbb{E}\left[\tilde{y}_i - \mathbb{E}[\tilde{y}_i]\right] = 0.$$

Similarly,

$$\mathbb{E}[AC] = (f(x_i) - \mathbb{E}[\tilde{y}_i])\,\mathbb{E}[\epsilon_i] = 0,$$

since

$$\mathbb{E}[\epsilon_i] = 0.$$

Finally,

$$\mathbb{E}[BC] = \mathbb{E}\left[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])\,\epsilon_i\right] = 0,$$

because the noise has zero mean and is assumed to be independent of the training set.

Therefore,

$$\boxed{\mathbb{E}\left[(y - \tilde{y})^2\right] = \frac{1}{n}\sum_{i=1}^{n}\left(f(x_i) - \mathbb{E}[\tilde{y}_i]\right)^2 + \frac{1}{n}\sum_{i=1}^{n}\mathrm{Var}(\tilde{y}_i) + \sigma^2}$$

or, equivalently,

$$

2 MSE

Bias^2 + Variance + Noise.

$$

2.1 Solution exercise 2b)

The term

$$\left(f(x_i) - \mathbb{E}[\tilde{y}_i]\right)^2$$

is the squared bias. It describes how well the average prediction of our model agrees with the true function. A large bias means that the mean prediction of the model is systematically far away from the true value.

The term

$$\mathrm{Var}(\tilde{y}_i) = \mathbb{E}\left[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2\right]$$

is the variance of our model. It tells us how much the fitted model changes between different training sets, or equivalently how spread out the predictions are around their mean.

Finally,

$$\mathbb{E}[\epsilon_i^2] = \sigma^2$$

is the irreducible noise. This contribution cannot be removed by changing the model, since it originates from the stochastic noise in the data.

2.2 Solution exercise 2c)

Suppose that

$$y_{\text{test},i} = f(x_i) + \epsilon_{\text{test},i},$$

where the test noise satisfies

$$\mathbb{E}[\epsilon_{\text{test},i}] = 0, \qquad \mathbb{E}[\epsilon_{\text{test},i}^2] = \sigma^2.$$

If we use the test data to estimate the squared bias, we obtain

$$\mathbb{E}\left[\left(y_{\text{test},i} - \mathbb{E}[\widetilde{y}_i]\right)^2\right].$$

Substituting

$$y_{\text{test},i} = f(x_i) + \epsilon_{\text{test},i},$$

gives

$$\mathbb{E}\left[\left(f(x_i) + \epsilon_{\text{test},i} - \mathbb{E}[\widetilde{y}_i]\right)^2\right].$$

Expanding,

$$= \left(f(x_i) - \mathbb{E}[\widetilde{y}_i]\right)^2 + 2\left(f(x_i) - \mathbb{E}[\widetilde{y}_i]\right)\mathbb{E}[\epsilon_{\text{test},i}] + \mathbb{E}[\epsilon_{\text{test},i}^2].$$

Since

$$\mathbb{E}[\epsilon_{\text{test},i}] = 0,$$

we get

$$\mathbb{E}\left[\left(y_{\text{test},i} - \mathbb{E}[\widetilde{y}_i]\right)^2\right] = \left(f(x_i) - \mathbb{E}[\widetilde{y}_i]\right)^2 + \sigma^2.$$

Thus, the bias measured using the test data contains an additional contribution from the noise:

$$

3 measured bias^2

true bias^{2+} 2.

$$

Therefore, if σ^2 is known, the true squared bias can be estimated as

$$\text{true bias}^2 = \text{measured bias}^2 - \sigma^2.$$

Using the bias-variance decomposition,

$$\mathbb{E}\left[(y_i - \tilde{y}_i)^2\right] = \text{Bias}^2 + \text{Variance} + \sigma^2.$$

Hence the irreducible noise σ^2 sets the upper limit on the error. As the bias approches the true bias the upper limit of the error

$$\boxed{\text{error} \geq \text{bias}^2 + \text{variance.}}$$

3.1 Exercise 3: The bias-variance tradeoff with the bootstrap

We use the same data as in the Tuesday session (and as in last week's exercise 3),

$$y = e^{-x^2} + 1.5\, e^{-(x-2)^2} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 0.1^2),$$

on $x \in [-3, 3]$.

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.utils import resample
from sklearn.metrics import mean_squared_error

np.random.seed(2026)
n = 40
x = np.linspace(-3, 3, n).reshape(-1, 1)
y = np.exp(-x**2) + 1.5 * np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.
 ↪shape)
```

1. Split the data in training and test parts (80/20). For polynomial degrees 0 to 13, use the bootstrap (100 resamples of the training data) to estimate the test error, the squared bias and the variance, and plot the three curves as functions of the degree (compare with `week36_bias_variance.pdf` from the lecture — with seed 2026 you should reproduce it).
2. Discuss the figure in terms of Eq. (2.47): where is the model dominated by bias, where by variance, and where would you place the optimal degree?
3. Repeat the analysis for $n = 100$ and $n = 400$ data points and comment on how the curves and the optimal degree move.
4. Replace OLS by Ridge regression at a fixed high degree (say 12) and study the three terms as functions of λ (for example `np.logspace(-6, 2, 9)`). Which term does λ attack, and at what cost?

[2]:
```
# Your code for exercise 3 here

def bias_variance(n, max_deg=13, n_bootstraps=100):
    """
    This function performs n_boostraps and computes the bias and
    variance for polynomial degree 0 to max_deg.
    """
    np.random.seed(2026)
    x = np.linspace(-3, 3, n).reshape(-1, 1)
    y = np.exp(-x**2) + 1.5 * np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.
 ↪shape)

    degrees = np.arange(max_deg + 1)

    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,␣
 ↪random_state=2026)

    error = np.zeros(max_deg + 1)
    bias = np.zeros(max_deg + 1)
    var = np.zeros(max_deg + 1)

    for degree in degrees:
        model = make_pipeline(
            PolynomialFeatures(degree=degree),
            LinearRegression(fit_intercept=False)
        )

        # one column for each bootstrap
        y_pred = np.zeros((len(y_test), n_bootstraps))

        # start resampling
        for i in range(n_bootstraps):
            x_b, y_b = resample(x_train, y_train)
            model.fit(x_b, y_b)
            y_pred[:,i] = model.predict(x_test).ravel()
```

```
        error[degree] = np.mean(
            np.mean((y_test - y_pred)**2, axis=1, keepdims=True)
            )

        bias[degree] = np.mean(
            (y_test - np.mean(y_pred, axis=1, keepdims=True))**2
            )
        var[degree] = np.mean(np.var(y_pred, axis=1, keepdims=True))

    return error, bias, var


def plot_bias_variance(n_values):
    """
    Plot bias-variance decomposition for one or multiple dataset sizes.

    Parameters:
    - n_values : int or list of ints      Number of data points. Example:  40 ␣
 ↪or  [40, 100, 400]
    """

    # Convert a single integer into a list
    if isinstance(n_values, int):
        n_values = [n_values]

    n_plots = len(n_values)

    if n_plots == 1:
        fig, ax = plt.subplots(figsize=(6.6, 4.0))
        axes = [ax]

    else:
        fig, axes = plt.subplots(1, n_plots, figsize=(5 * n_plots, 4),␣
 ↪sharey=True)

    for ax, n in zip(axes, n_values):
        error, bias, var = bias_variance(n=n)
        degrees = np.arange(len(error))

        ax.plot(degrees, error, "o-", color="midnightblue", label="Test error")
        ax.plot(degrees, bias, "s-", color="mediumvioletred", label=r"Bias$^2$")
        ax.plot(degrees, var, "d-", color="chocolate", label="Variance")

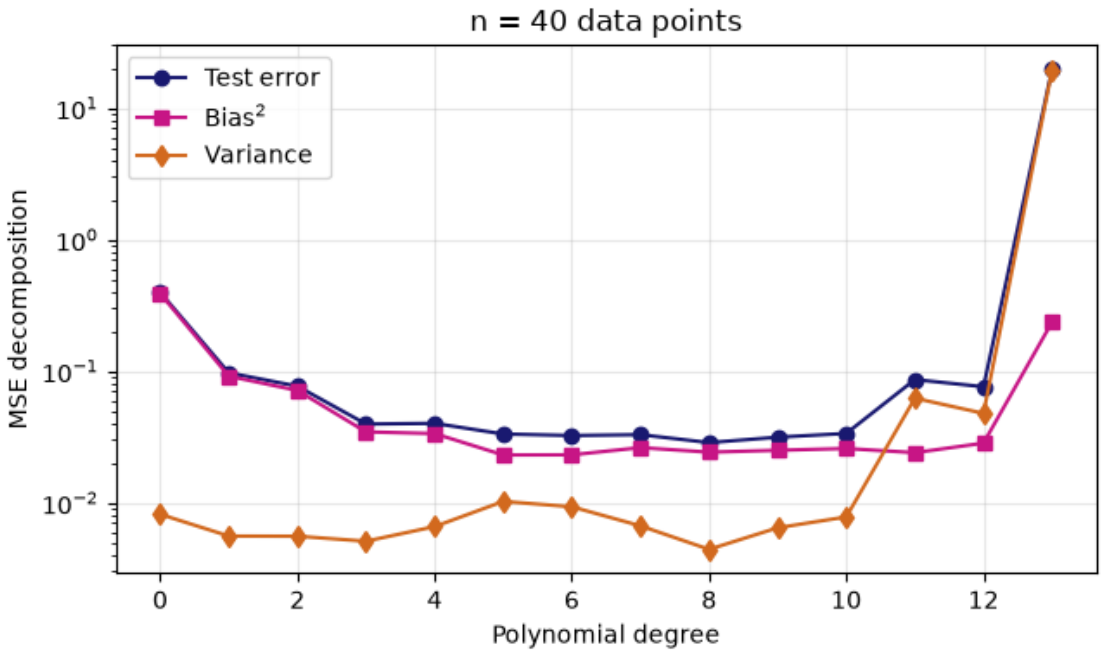
        ax.set_yscale("log")
        ax.set_xlabel("Polynomial degree")
        ax.set_title(f"n = {n} data points")
```

```
        ax.grid(alpha=0.3)

    axes[0].set_ylabel("MSE decomposition")
    axes[0].legend()

    plt.tight_layout()
    plt.show()
```

[3]:
```
# for n = 40:
plot_bias_variance(40)
```



2) Discussion:

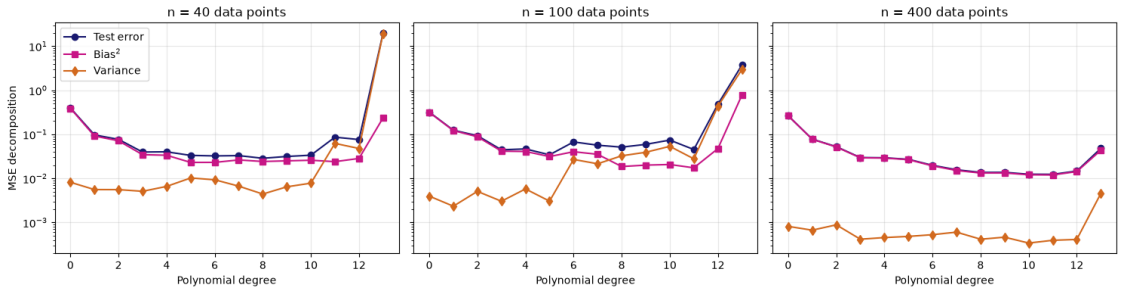
The model dominated by bias for low polynomial degrees, approximately $0 \leq d \leq 3$. This is where the bias dominates the MSE, meaning that the model is too simple to reproduce the original function. This is underfitting.

The model is dominated by variance for higher polynomial degrees, approximately $d \geq 11$. For these degrees the variance become very high and it dominates the test error. This indicates that the model is overfitted.

For intermediate polynomial degrees, approximately $4 \leq d \leq 11$, the bias is reduced and both the bias and the variance is relatively small. This indicates that these degrees are sufficient for the model, illustarting the bias-variance tradeoff.

The optimal polynomial degree is naturally at near the minima of the test eroor, so around $d \approx 8$, where both the bias and the variance is relatively low.

[4]:
```
# for n = [40, 100, 400]:
plot_bias_variance([40, 100, 400])
```



3) Discussion

We see that when we increase the number of data points the bias is roughly unchanged while the variance of the model reduces. For $n = 40$ the variance increases notably for higher polynomial degrees, with $d \approx 8$ giving the best fit. For $n = 100$ the higher degree models are more stable and the minima of the test error moves to higher polynomial degrees, with the optimal degree being around $d \approx 11$. For $n = 400$ data points, the variance is substantially decreased compared to the previous cases and remains small across almost all polynomial degrees. The bias dominates the test error in this case. The minima is shifted to $d \approx 12$.

Consequently, increasing the amount of training data allows us to use more complex models without without being penalized by variance.

[5]:
```
# using Ridge regression at d = 12

def ridge_bias_variance(n, degree=12, n_bootstraps=100):

    np.random.seed(2026)
    x = np.linspace(-3, 3, n).reshape(-1, 1)
    y = np.exp(-x**2) + 1.5 * np.exp(-(x - 2)**2) + np.random.normal(0, 0.1, x.
 ↪shape)

    x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,␣
 ↪random_state=2026)

    lambdas = np.logspace(-6, 2, 9)
    error = np.zeros(len(lambdas))
    bias = np.zeros(len(lambdas))
    var = np.zeros(len(lambdas))

    for j, lmbda in enumerate(lambdas):
        model = make_pipeline(
            PolynomialFeatures(degree=degree),
            Ridge(alpha=lmbda, fit_intercept=False)
```

```
        )

        # one column for each bootstrap
        y_pred = np.zeros((len(y_test), n_bootstraps))

        # start resampling
        for i in range(n_bootstraps):
            x_b, y_b = resample(x_train, y_train)
            model.fit(x_b, y_b)
            y_pred[:,i] = model.predict(x_test).ravel()

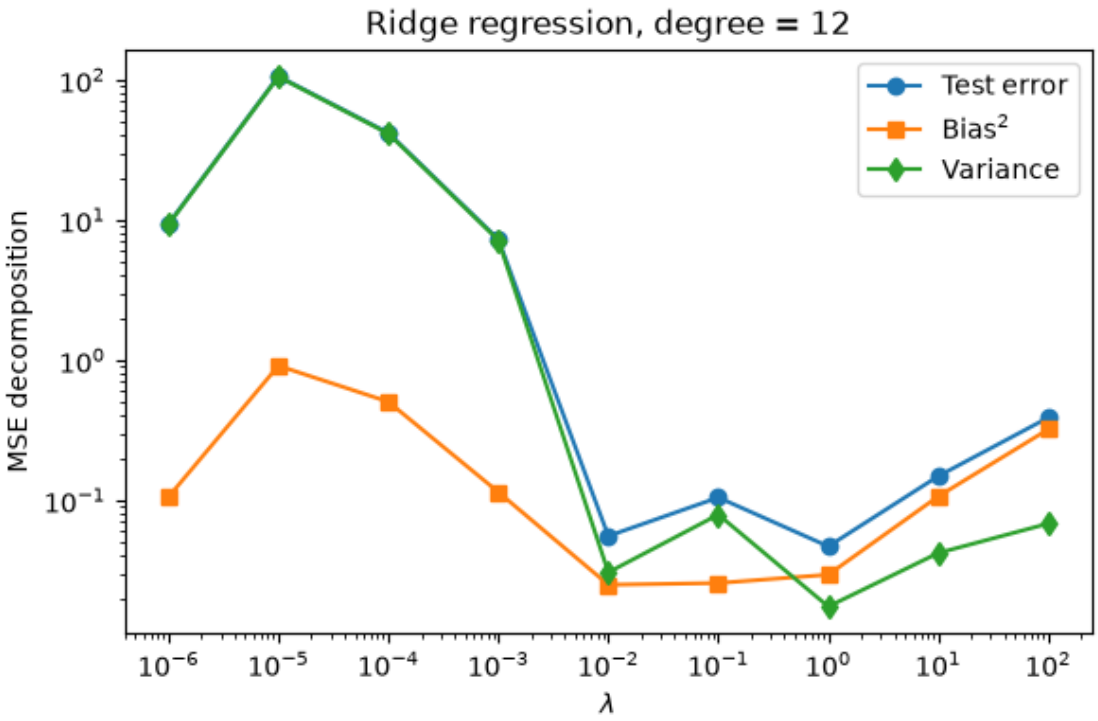
        error[j] = np.mean(
            np.mean((y_test - y_pred)**2, axis=1, keepdims=True)
            )

        bias[j] = np.mean(
            (y_test - np.mean(y_pred, axis=1, keepdims=True))**2
            )
        var[j] = np.mean(np.var(y_pred, axis=1, keepdims=True))

    return lambdas, error, bias, var


# plotting
lambdas, error, bias, var = ridge_bias_variance(n=40)

plt.figure(figsize=(6.6, 4.0))
plt.plot(lambdas, error, "o-", label="Test error")
plt.plot(lambdas, bias, "s-", label=r"Bias$^2$")
plt.plot(lambdas, var, "d-", label="Variance")
plt.xscale("log")
plt.yscale("log")
plt.xlabel(r"$\lambda$")
plt.ylabel("MSE decomposition")
plt.title("Ridge regression, degree = 12")
plt.legend()
plt.show()
```



4) Discussion

For small regularization, aproximately $10^{-6} \leq \lambda \leq 10^{-3}$ the variance is very large and domantes the test error. As the regularization increases, the variance decreases because Ridge suppresses large coefficients. At degree 12, the model is relatively flexible, and we saw earlier that such a complex model can have high variance unless we use a larger training data set.

For $\lambda \approx 10^{-2}$ the variance, bias and test error decreases substantially. For larger vlaues of λ, the model becomes increasingly regularized and the bias increases because coefficients are driven closer to zero, reducing the models flexibility. Thus, the regularization parameter λ mainly attacks the varaince of the model, but the cost of this is an increase in bias for large regularization parameters.

From the figure, $\lambda \approx 1$ is the optimal regularization as the test error is at a minima and the variance is lowest here.

3.2 Exercise 4: Cross-validation

Still with the same type of data, we now select models without consuming a test set.

1. Use scikit-learn's (or perhaps write your own) k-fold cross-validation function: shuffle the data, split into k folds, use each fold as test set once, and return the mean and standard deviation over folds of the test MSE. Do **not** use `cross_val_score` inside your own function.
2. Use it with $k = 5$ to compute the CV estimate of the MSE for OLS with polynomial degrees 0 to 13, and compare with the bootstrap analysis of exercise 3: do the two methods point to the same degree window?

3. For a degree-6 polynomial with Ridge regression on $y = 3x^2 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 1)$, $n = 100$ (Tuesday's Case 2), compute the CV MSE on `np.logspace(-3, 5, 100)` and find the optimal λ. Any scaling/preprocessing must be fitted **inside** each fold — explain in one or two sentences why.
4. Try $k = 5, 10$ and n (leave-one-out). Comment on the stability of the selected λ and on the computational cost.

3.3 4.1 Solution

[6]:
```
def kfold_cv(X, y, model_degree, k=5, ridge = False, lam = 0):
    kf = KFold(n_splits=k, shuffle=True, random_state=2026)
    mse_list = []

    for train_idx, test_idx in kf.split(X):
        if not ridge:
            model = make_pipeline(PolynomialFeatures(model_degree),␣
↪LinearRegression())
        else:
            model = make_pipeline(PolynomialFeatures(model_degree), Ridge(lam))
        model.fit(X[train_idx], y[train_idx])
        y_actual, y_pred = y[test_idx], model.predict(X[test_idx])
        mse_list.append(mean_squared_error(y_actual, y_pred))

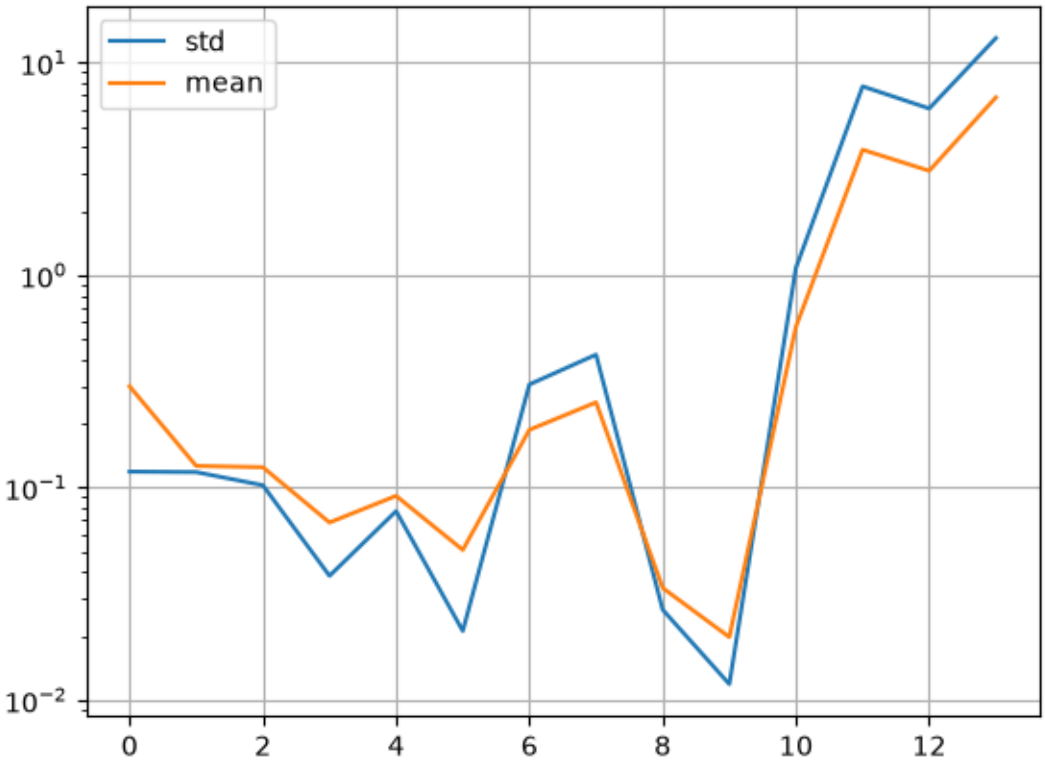
    return np.mean(mse_list), np.std(mse_list)
```

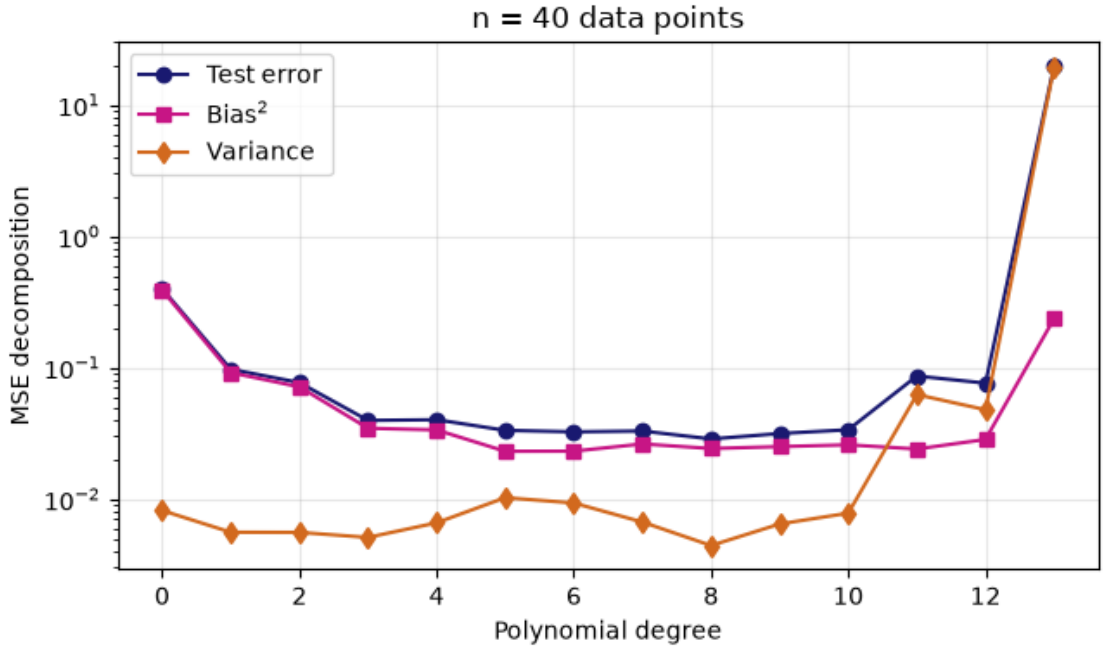
3.4 4.2 Solution

[7]:
```
degrees = np.arange(0, 14, 1)
k = 5
mean_mse, std_mse = np.zeros(len(degrees)), np.zeros(len(degrees))

for i in range(len(degrees)):
    mean_mse[i], std_mse[i] = kfold_cv(x, y, degrees[i], k)

plt.plot(degrees, std_mse, label="std")
plt.plot(degrees, mean_mse, label="mean")
plt.yscale('log')
plt.grid(True)
plt.legend()
plt.show()
plot_bias_variance(40)
```





The CV and bootstrap methods diverge around degrees 6-8, suggesting they my select diffrent

optimal degrees. however, both methods agree on the overall trend: bias dominates at low degrees and variance dominates at high degrees. The key diffrence is which specific degree minimizes error.

3.5 4.3 Solution

[8]:
```
n = 100
x = np.linspace(-3, 3, n).reshape(-1, 1)
y = 3 * x**2 + np.random.normal(0, 1, x.shape)

lambdas = np.logspace(-3,5,100)
mean_mse, std_mse = np.zeros(len(lambdas)), np.zeros(len(lambdas))

for i in range(len(lambdas)):
    mean_mse[i], std_mse[i] = kfold_cv(x, y, 6, 5, True, lambdas[i])


print(f"Optimal lambda: {lambdas[np.argmin(mean_mse)]}")
print(f"Minimum CV MSE: {np.min(mean_mse):.6}")
```

```
Optimal lambda: 0.26560877829466867
Minimum CV MSE: 0.970893
```

Fitting preprocessing inside each fold prevents data leakage. The test set must be treated as unseen data, so the scalar should only learn for the training fold's statistics, not for the test fold.

3.6 4.4 Solution

[9]:
```
k = [5, 10, 100]

for j in k:
    for i in range(len(lambdas)):
        mean_mse[i], std_mse[i] = kfold_cv(x, y, 6, j, True, lambdas[i])


    print(f"Optimal lambda for k = {j}: {lambdas[np.argmin(mean_mse)]} \t␣
 ↪number of fits: {j*len(lambdas)}")
```

```
Optimal lambda for k = 5: 0.26560877829466867    number of fits: 500
Optimal lambda for k = 10: 0.22051307399030456   number of fits: 1000
Optimal lambda for k = 100: 0.26560877829466867          number of fits: 10000
```

The three optimal lambdas is: - k = 5: 0.811 - k = 10: 0.559 - k = 100: 0.673 Which is relativly stable across diffrent k values, this suggests the choise of k does not drastically change which lambda is selected. Computational cost grows substationally with increesed k.

3.7 Exercise 5 (optional): error bars from the bootstrap

Use the bootstrap to attach uncertainties to fitted parameters, where the analytical formula of exercise 1 can be checked — and where it cannot.

1. For OLS on a degree-3 polynomial fit of the exercise-3 data, bootstrap the training data 1000 times and histogram the resulting $\hat{\theta}_j^*$. Compare the bootstrap standard deviations with the analytical standard errors $\hat{\sigma}\sqrt{[(X^TX)^{-1}]_{jj}}$ of Eq. (2.43).
2. Repeat for Lasso with a small penalty. Why is the bootstrap now the only option on the table? (Which assumption behind Eq. (2.43) fails?)

[ ]:

[10]:
```
# Your code for exercise 5 here
```

[ ]:

3.7.1 Practicalities

Deliver your solutions (a single Jupyter notebook, with the analytical parts either typeset in Markdown/LaTeX or as a scanned handwritten note) in Canvas before **Friday September 4 at midnight**. Collaboration is encouraged — list your collaborators.